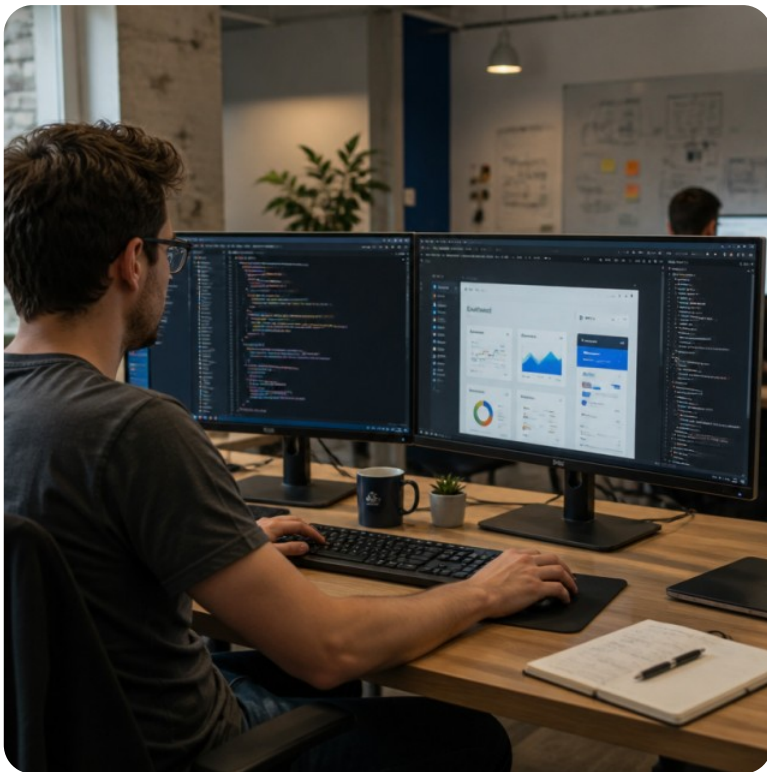


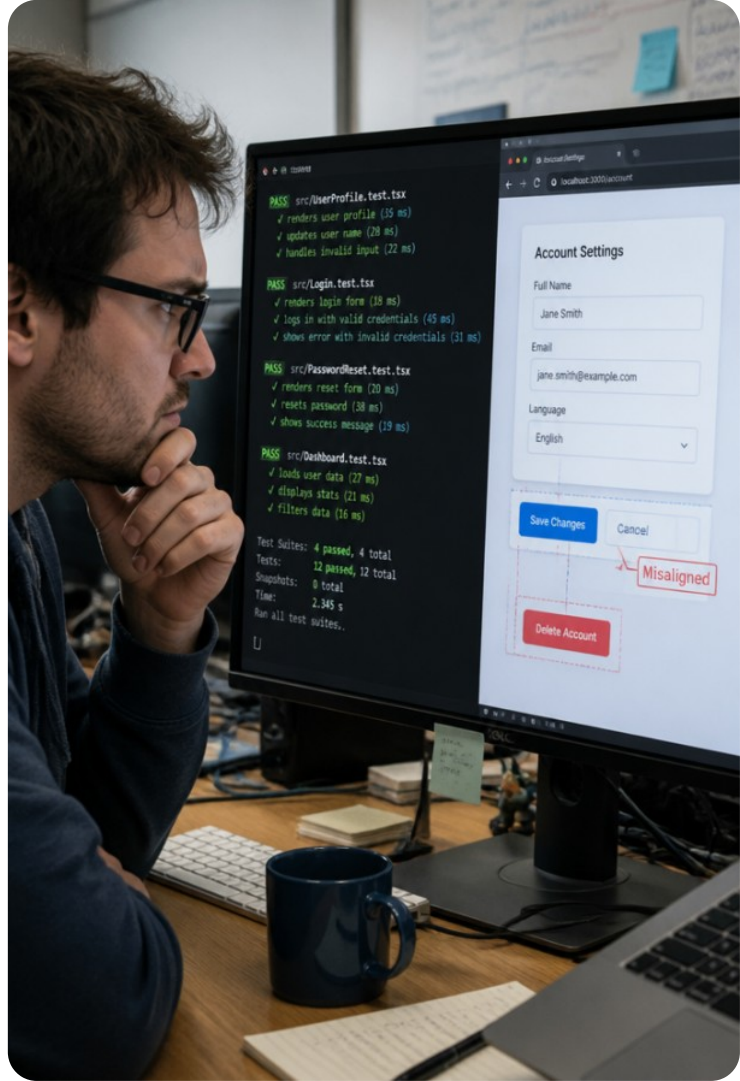


Testes de Integração com DOM usando Testing Library

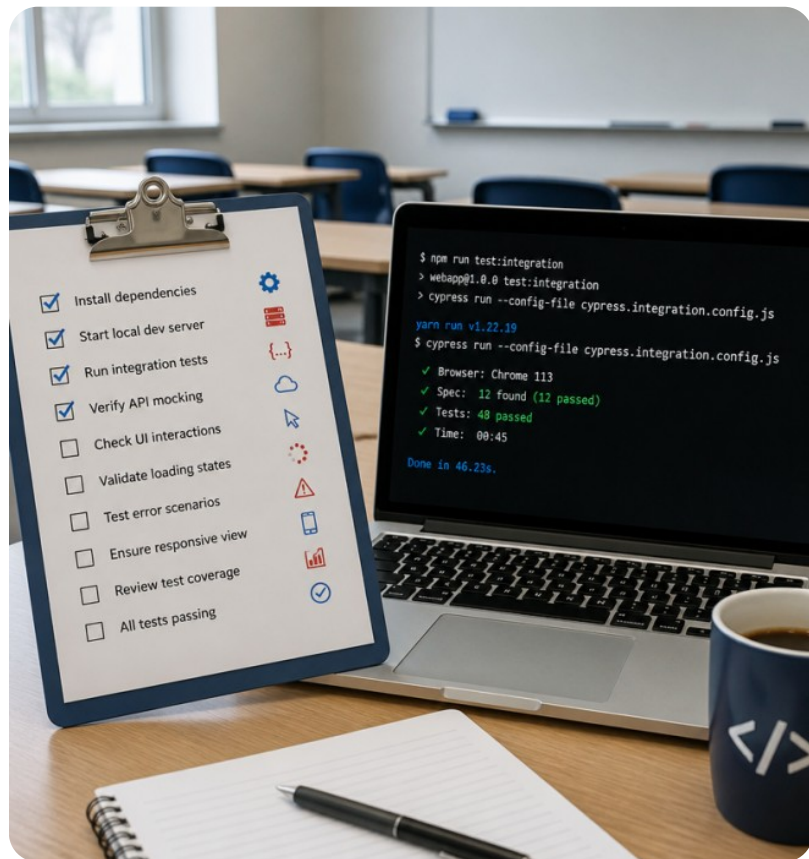
Validando interfaces web com foco na experiência real do usuário.

O Teste Passou, mas a Tela Quebrou?

Se o teste valida variáveis internas mas o usuário não consegue clicar no botão, o teste realmente funcionou?



Objetivos da Aula



O que vamos dominar hoje

Nesta aula do Bloco 04, você desenvolverá três competências essenciais:

Compreender a **filosofia da Testing Library** centrada no comportamento do usuário

Aplicar a **hierarquia de queries** acessíveis para localizar elementos no DOM

Escrever **testes de integração** completos para botões e formulários com Vitest

Vocabulário Fundamental



Integração -
Validação de
múltiplos nós do
DOM e eventos
trabalhando em
conjunto.



Elemento - Nó
visual do
documento HTML,
como botões,
títulos ou inputs.



Interação - Ação
intencional do
usuário, como
digitação, cliques
ou navegação via
teclado.



Acessibilidade -
Garantia de uso da
interface por
leitores de tela e
tecnologias
assistivas.

Recapitulando: Vitest e Padrão AAA

FUNDAMENTOS

No Bloco 03, vimos que testes seguem três etapas:

Arrange: preparamos cenário, dados e renderização

Act: executamos a ação do componente ou usuário

Assert: verificamos o resultado com

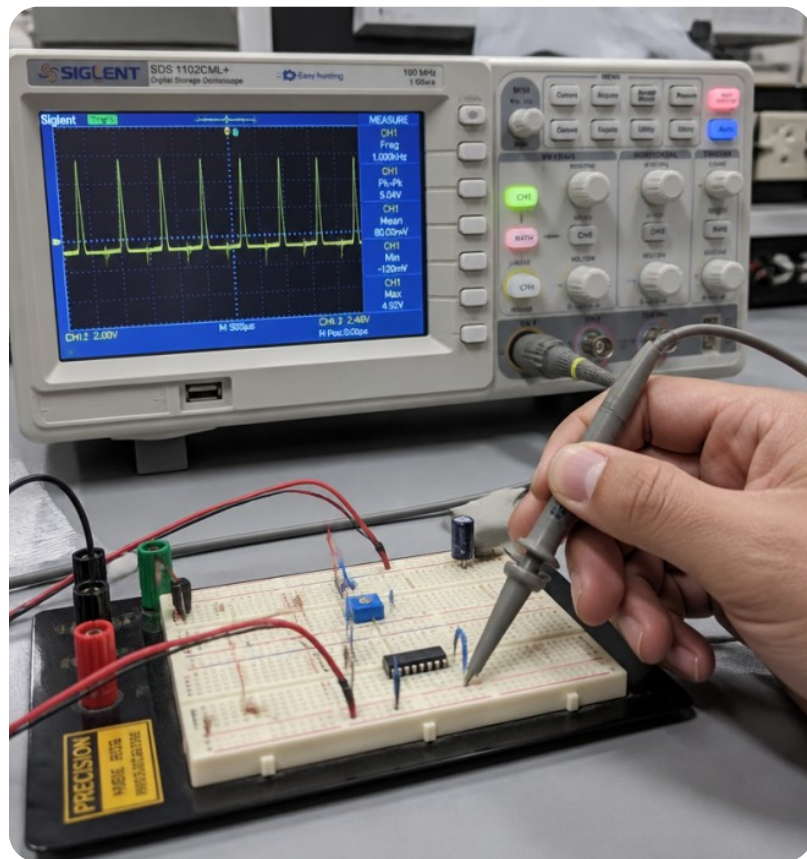


Lembre-se

O padrão AAA mantém testes organizados e legíveis.



Recapitulando: Mocks e Spies no Vitest



FUNDAMENTOS

Funções falsas e observadores permitem isolar dependências externas durante os testes:

- : gera funções simuladas para capturar argumentos e contagem de execuções
- : monitora comportamentos em objetos existentes sem destruí-los



Ponto-chave

Usamos para garantir que cliques chamem callbacks esperados.

Quiz de Revisão: Vitest e AAA



Pergunta 1:

Qual é a função da etapa 'Arrange' no padrão estrutural AAA?

Pergunta 2:

No Vitest, qual método cria uma função mockada para monitorar chamadas?

Pergunta 3:

Qual matcher valida se uma função simulada foi disparada com sucesso?

Quiz de Revisão: Vitest e AAA



Resposta 1:

Configurar o estado inicial, preparar variáveis e montar o ambiente do teste.

Resposta 2:

`vi.fn()`

Resposta 3:

`toHaveBeenCalled()` ou `toHaveBeenCalledTimes()`



A Filosofia da Testing Library

Kent C. Dodds definiu o princípio orientador da biblioteca:

Quanto mais os seus testes se assemelharem à forma como o software é utilizado, mais confiança eles poderão lhe dar.



Ponto-chave

O usuário não inspeciona variáveis internas nem classes privadas; ele interage com textos, botões e campos visíveis.

Implementação versus Comportamento



Detalhes de Implementação

Checa interno

- Valida métodos privados
- Usa seletores frágeis (.btn-primary)
- Quebra em refatorações



Comportamento Visível

Usa ('Incrementar')

- Simula interação real
- Valida 'Total: 1' na tela
- Resiste a mudanças internas

Por que Evitar Detalhes Internos?

Testar a estrutura interna gera dois problemas clássicos:

Falsos positivos: teste passa, mas o botão está invisível ao usuário

Falsos negativos: mudamos uma variável interna, a interface segue perfeita, mas o teste falha



Atenção

Testes frágeis atrasam entregas ao exigir correções em refatorações cosméticas.



Instalação do Ambiente

Preparando o Ecossistema

Para executar testes de DOM integrados ao Vitest, instalamos os pacotes oficiais via terminal:

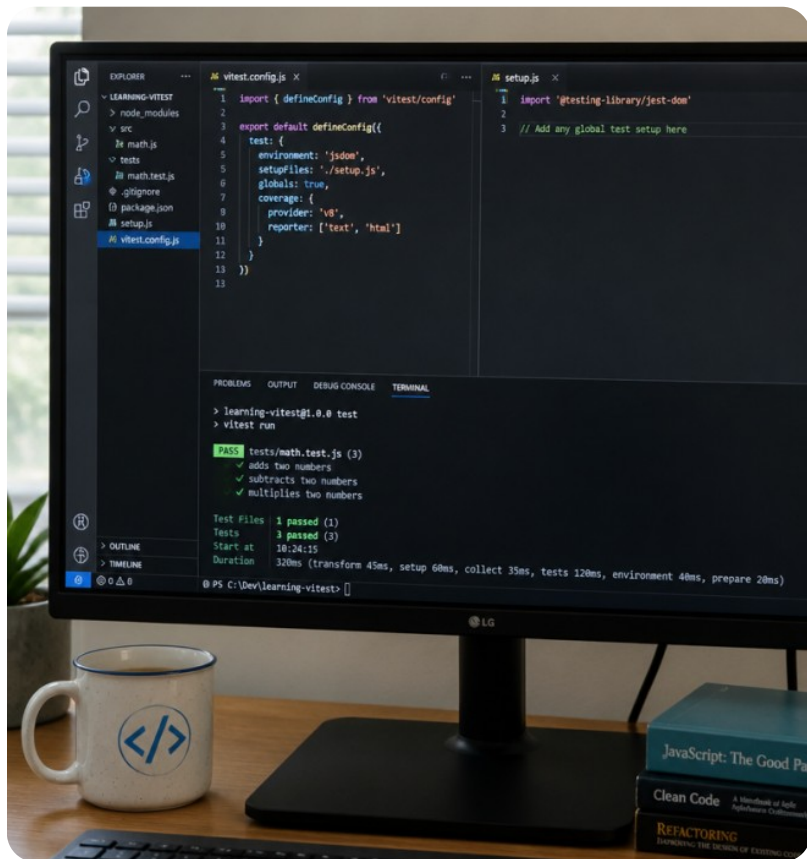
- : núcleo de busca e interação no DOM
- : matchers semânticos personalizados
- : simulação realista de eventos do usuário
- : implementação do DOM em Node.js para emular o navegador



Exemplo

Execute no terminal: `npm install -D @testing-library/dom @testing-library/jest-dom @testing-library/user-event jsdom`

Configurando Vitest com JSDOM



O Vitest precisa emular as APIs do navegador durante a execução dos testes:

Definimos em

Declaramos para carregar matchers customizados

No setup, importamos para estender o



Lembre-se

O JSDOM recria , e eventos DOM dentro do ambiente Node.

Hierarquia de Queries

Testing Library prioriza seletores acessíveis:

- 1 `getByRole`: prioridade máxima; busca papéis e nomes acessíveis.
- 2 `getByLabelText`: ideal para inputs associados a labels.
- 3 `getByPlaceholderText`: alternativa para inputs sem label.
- 4 `getByTestId`: último recurso para elementos sem significado semântico.

Prioridade 1: getByRole

QUERIES

O método reflete diretamente a árvore de acessibilidade da interface:

Localiza botões (), títulos (), caixas de texto () e links ()

- Permite refinar pelo nome acessível com a opção
- Garante que a interface seja operável por leitores de tela

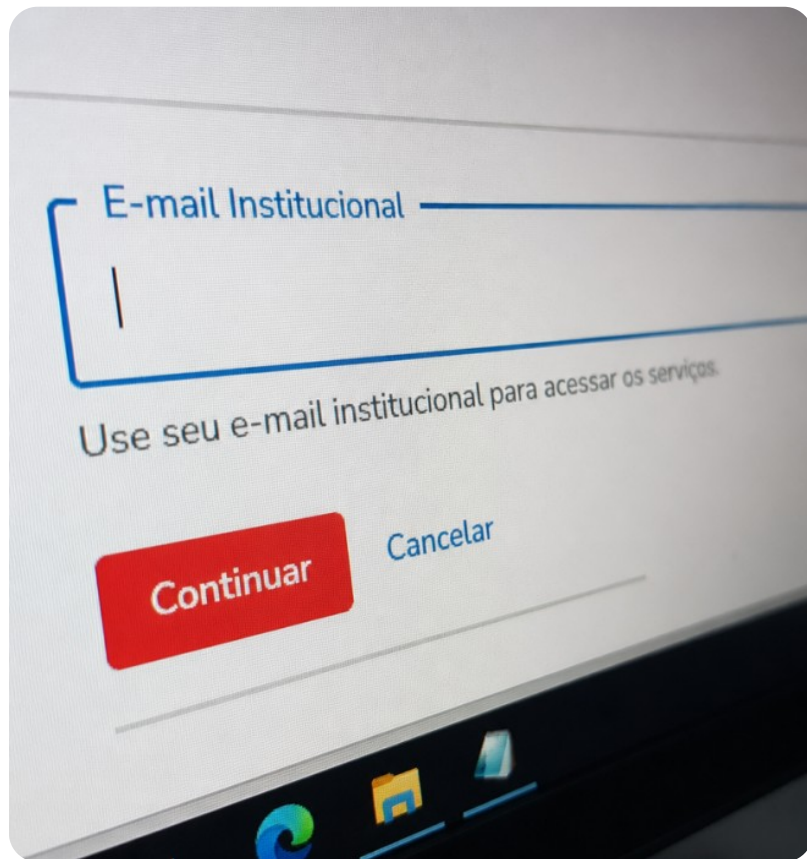


Exemplo

```
screen.getByRole('button', { name:  
  /enviar dados/i })
```



Prioridade 2 e 3: Labels e Placeholders



QUERIES

Ao interagir com campos de entrada, seguimos as convenções naturais do usuário:
: seleciona o campo associado ao texto do , simulando quem lê o formulário

- : útil para caixas de pesquisa onde o placeholder orienta a ação imediata



Ponto-chave

Associar corretamente o do label ao do input viabiliza o teste e a acessibilidade.

getById: O Último Recurso

QUERIES

O seletor serve apenas quando nenhuma abordagem semântica for viável:

- Use em elementos visuais, contêineres dinâmicos ou SVGs complexos
- O usuário real não enxerga atributos
- O excesso de test-ids mascara problemas de semântica e acessibilidade



Atenção

Evite usar data-testid quando houver roles ou labels disponíveis.



Tipos de Busca: getBy, queryBy e findBy



getBy*

- Retorna o nó

Erro se ausente/duplicado

- Uso: verificar presença

queryBy*

Retorna nó ou **null**

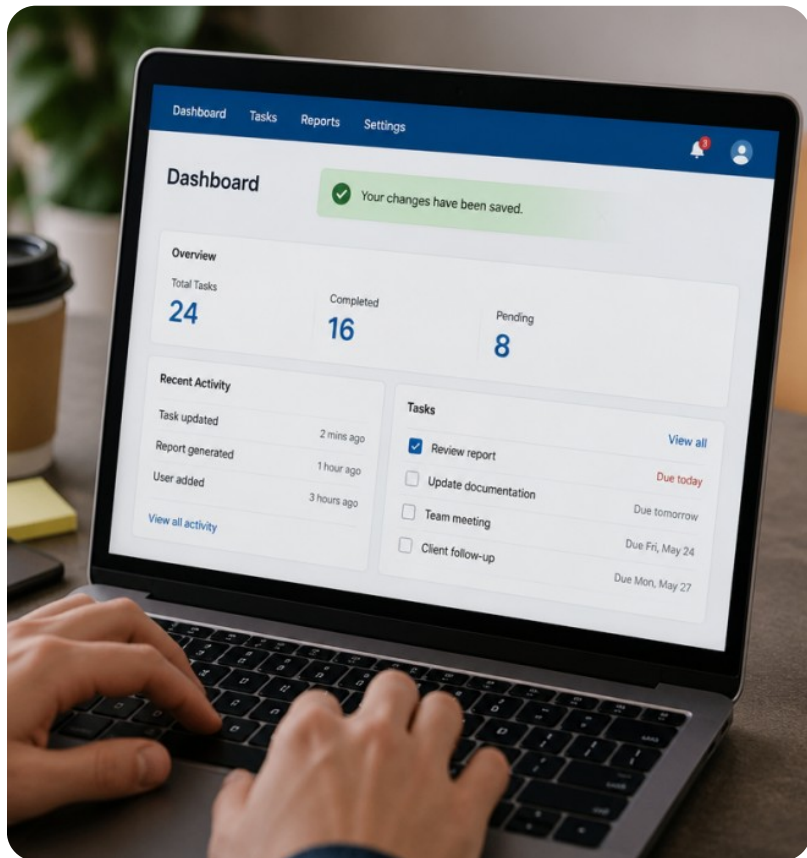
- Sem erro se ausente
- Erro se duplicado
- Uso: validar ausência

findBy*

Retorna **Promise**

- Aguarda (até 1s)
- Combina e
- Uso: itens assíncronos

Validando Ausência com queryBy



Quando queremos comprovar que uma mensagem de erro sumiu, o causaria falha precoce:

O explode a execução ao não encontrar o item

O retorna , permitindo a asserção correta

- Estrutura comum:



Lembre-se

Para verificar se um elemento sumiu da tela, sempre utilize queryBy em conjunto com .not.

Buscando Elementos Assíncronos com findBy

Interfaces web modernas carregam dados sob demanda após ações do usuário:

- aguarda o elemento ser renderizado
- Se o nó não surgir dentro do tempo limite padrão, o teste falha com relatório descritivo
- Reduz testes instáveis e elimina esperas arbitrárias com



Ponto-chave

Funções que utilizam `findBy` devem ser sempre declaradas com `async` e invocadas com `await`.



Verificação: Seleção de Queries

Qual método de busca deve ser utilizado para validar que uma janela modal NÃO está presente na tela?

1. `getByTestId('dialog')`, porque ignora a hierarquia de acessibilidade
2. `getByRole('dialog')`, pois ele emite um alerta seguro sem quebrar o teste
3. `queryByRole('dialog')`, pois retorna null sem lançar erro ao não encontrar
4. `findByRole('dialog')`, porque aguarda a exclusão do elemento no DOM

Verificação: Seleção de Queries



Qual método de busca deve ser utilizado para validar que uma janela modal NÃO está presente na tela?

1. `getByTestId('dialog')`, porque ignora a hierarquia de acessibilidade
2. `getByRole('dialog')`, pois ele emite um alerta seguro sem quebrar o teste
3. `queryByRole('dialog')`, pois retorna null sem lançar erro ao não encontrar
4. `findByRole('dialog')`, porque aguarda a exclusão do elemento no DOM

Simulando Ações com user-event

INTERAÇÃO

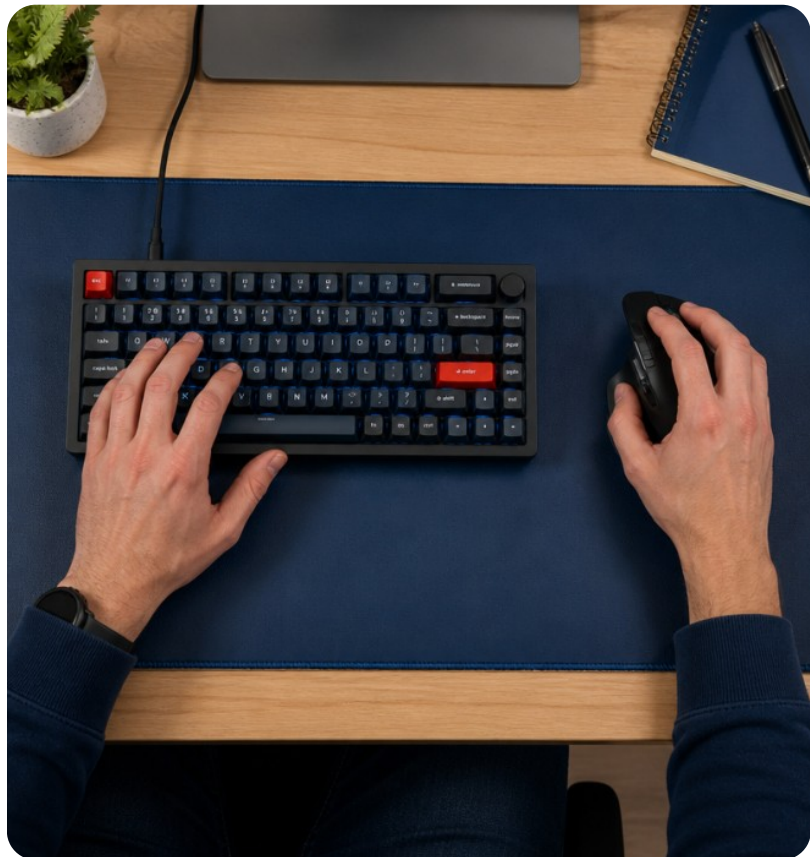
O pacote reproduz com precisão o comportamento no navegador:

- : dispara eventos brutos e sintéticos diretamente no elemento
- : simula o ciclo completo com hover, mousedown, focus, mouseup e click
- Exige inicialização prévia:



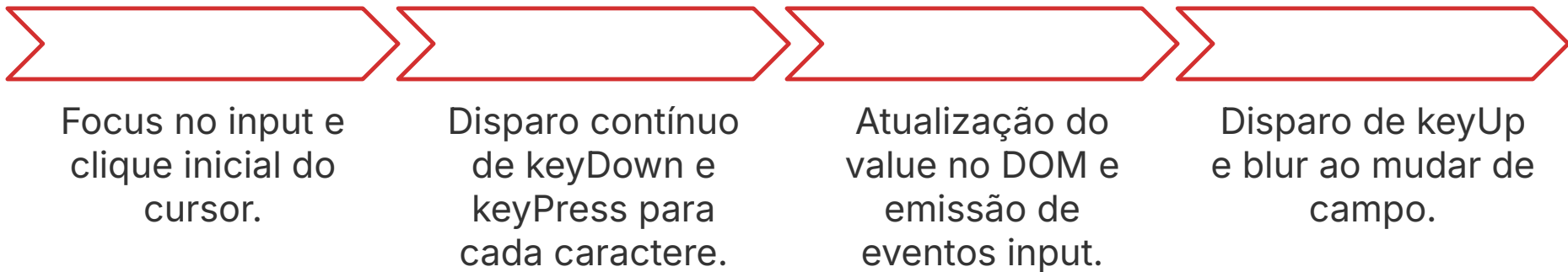
Ponto-chave

Sempre prefira `userEvent` a `fireEvent` para simular interações fidedignas com o DOM.



Ciclo de Interação Realista

Quando um usuário digita em um campo de texto, o navegador aciona uma sequência encadeada de eventos que o reproduz integralmente:



Acessibilidade Implícita via ARIA Roles



Ao priorizar , você avalia acessibilidade continuamente:

Se o teste não consegue achar um botão por , um leitor de tela também não conseguirá

Substitui divs clicáveis improvisadas por elementos semânticos genuínos ()

Incentiva uso correto de atributos como e



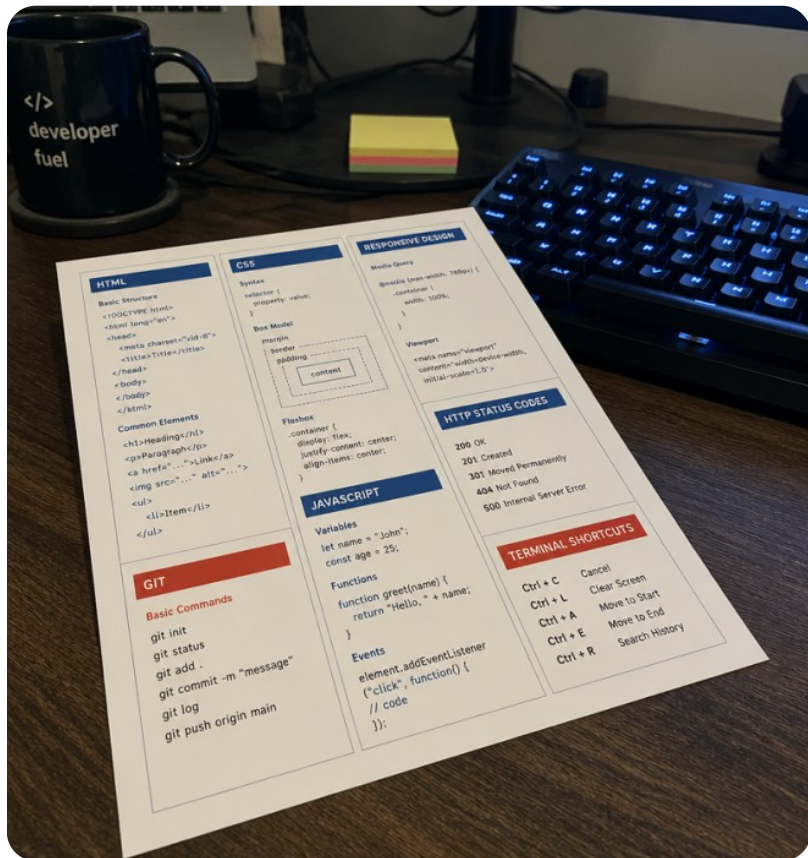
Curiosidade

Bons testes de integração servem como um auditor automático de conformidade WCAG.

Matchers do jest-dom

A extensão adiciona asserções expressivas ao :

- : valida presença física na árvore DOM
- : confirma que o nó não possui ou opacidade zero
- e : inspecionam o estado de botões e campos
- : confere o conteúdo digitado em caixas de entrada



Demonstração: Testando um Botão Interativo



Passo a Passo: Teste do Botão

Estrutura do Teste

O que está acontecendo?

Inicializamos o para registrar cliques realistas

Criamos um spy com para monitorar a função de callback

- Localizamos o elemento sem depender de classes CSS frágeis
- Verificamos a execução da ação de negócio com precisão

Prática 1: Teste de Botão com Estados

Escreva uma suíte de testes com Vitest e Testing Library para um componente 'FollowButton' que alterna entre os estados 'Seguir' e 'Seguindo'.

Requisitos do exercício:

- 1. Arrange: Renderize o botão com o estado inicial ('Seguir').
- 1. Act: Execute o clique no botão usando `userEvent`.
- 1. Assert: Valide que a função callback foi chamada e que o rótulo do botão alterou para 'Seguindo' via `getByRole`.
- 1. Bônus: Valide que o botão possui o atributo `aria-pressed` correspondente.

Análise do Formulário com Validação

Formulários combinam múltiplos elementos que precisam cooperar harmonicamente:

Rótulos explícitos conectados aos inputs por atributos e

- Feedback visual dinâmico com mensagens de alerta quando o campo for inválido
- Botão de envio desabilitado até que todos os critérios estejam satisfeitos



Exemplo

Exemplo: digitar um e-mail sem '@' faz surgir a mensagem 'Formato inválido'.

The image shows a 'Register' form titled 'Create your account'. At the top, a red-bordered box contains the message: 'Please correct the errors below.' Below this, there are four input fields, each with a 'Linked label' pointing to it from the left. The first field is 'Full name *' with the value 'Alex Johnson'. The second field is 'Email address *' with the value 'alex.johnson'; a red error message 'Please enter a valid email address.' is shown below it, with a 'Validation error' label pointing to it from the right. The third field is 'Password *' with masked characters '....'; a red error message 'Password must be at least 8 characters.' is shown below it, with a 'Validation error' label pointing to it from the right. The fourth field is 'Confirm password *' with masked characters '...'; a red error message 'Passwords do not match.' is shown below it, with a 'Validation error' label pointing to it from the right. Below the input fields, there is a checkbox labeled 'I agree to the Terms of Service and Privacy Policy.' and a blue 'Create account' button. A 'Submit button' label points to the 'Create account' button from the right. At the bottom, there is a link: 'Already have an account? Log in'.

Cenário de Teste: Validação em Tempo Real



Renderizar o formulário e localizar input com `getByLabelText`.



Digitar e-mail incompleto com `await user.type(input, 'aluno')`.



Verificar se mensagem surge com `expect(screen.getByText(/inválido/i)).toBeVisible()`.



Completar com '@senai.br' e validar que a mensagem desapareceu via `queryBy`.

Prática 2: Teste de Formulário Completo



Teste de integração: Cadastro de usuário

- 1. Localize 'Nome' e 'Senha' com `getByLabelText`.
- 1. Localize botão 'Cadastrar' via `getByRole`.
- 1. Digite senha < 6 caracteres com `user.type()`.
- 1. Valide alerta: 'A senha deve ter no mínimo 6 dígitos'.
- 1. Corrija a senha e verifique se o botão habilita.

Revisão de Código e Feedback Visual

QUALIDADE

Se um teste falha, a Testing Library exibe o DOM no console:

- imprime o HTML atual
- Erros listam roles disponíveis se a query falhar
- Isso agiliza o debug sem abrir o browser



Ponto-chave

Erros da Testing Library sugerem queries acessíveis.



Armadilhas Comuns no DOM Testing

QUALIDADE

Evite erros comuns em testes de integração:

Esquecer em ou

Usar para verificar ausência de nós

- Depender de classes CSS que mudam em refatorações



Atenção

Se a classe CSS mudar mas a função persistir, o teste não deve quebrar.



Resumo dos Conceitos da Aula



Filosofia e Queries

- Teste a tela como o usuário real interage

Priorize e

- Garanta acessibilidade implícita aos componentes



Busca e Interação

para ausência e para assincronismo

- para disparar eventos realistas
- Matchers semânticos com

Desafio: O Fluxo Perfeito de Teste

Ordene as quatro etapas cronológicas para testar a submissão de um formulário:

**Localizar o input pelo seu rótulo semântico com
`getByLabelText`**

**Configurar o `userEvent` e renderizar os elementos no
`JSDOM`**

**Simular a digitação realista e o clique com `user.type` e
`user.click`**

**Validar a mensagem de sucesso com
`expect(...).toBeInTheDocument()`**

Desafio: O Fluxo Perfeito de Teste

Ordene as quatro etapas cronológicas para testar a submissão de um formulário:



1
Configurar o `userEvent` e renderizar os elementos no JSDOM

2
Localizar o input pelo seu rótulo semântico com `getByLabelText`

3
Simular a digitação realista e o clique com `user.type` e `user.click`

4
Validar a mensagem de sucesso com `expect(...).toBeInTheDocument()`

Próximo Passo: APIs e Estados Complexos

No Bloco 05, levaremos a integração para o próximo nível conectando múltiplos componentes com chamadas a APIs simuladas pelo Mock Service Worker.

